SpaceScope: A Big Data Analytics, Operational Lakehouse, and Predictive Machine Learning Framework for Global Space Missions

Sakshi Naikwade Sneha Supe Pranav Mode

Department of Computer Science and Engineering

MIT World Peace University, Pune, India

Project Guide: Dr. Varsha Powar

Big Data Technologies (BDT) Project • Implementation, Operational Lakehouse & Empirical Evaluation

Abstract

Historical records of orbital launch attempts provide vital empirical foundations for analyzing aerospace reliability, commercial expansion, and international participation. However, existing analyses remain predominantly retrospective, rely on opaque data preparation pipelines, lack operational mutation capabilities, and provide no integrated machine learning risk assessment. This paper presents **SpaceScope**, an end-to-end Big Data Technologies (BDT) framework combining: (i) an automated distributed Lakehouse pipeline ingesting and standardizing 4,324 historical orbital missions (1957–2020) across 4 execution partitions in Apache Spark (PySpark); (ii) an operational **Mission Operations Board** delivering complete Create, Read, Update, and Delete (CRUD) flight-log capabilities with synchronous analytical recalibration and search-aware export streaming; (iii) a **BDT Spark Engine & Benchmark Lab** evaluating empirical distributed scaling (Table VII) between single-node Pandas and 4-partition PySpark across 1× (4,324 records) and 5× (21,620 records) datasets, visualising Catalyst DAG execution plans, and mapping the 5 V's of aerospace Big Data; and (iv) an in-flight **Machine Learning Regression & Probabilistic Risk Engine** utilizing empirical Bayesian hierarchical smoothing to calculate real-time flight success and failure probabilities as parameters are entered. Empirical evaluation demonstrates 100% validity across timestamp and multi-part location parsing, exact elimination of business duplicates, sub-second preprocessing (1.04 s), sub-millisecond REST query latencies (0.16 ms), and sub-millisecond ML inference (< 1.0 ms) validated across 38 automated test suites.

Keywords—Big Data Technologies, Space Missions, Apache Spark, PySpark, Lakehouse Architecture, Machine Learning Regression, Bayesian Estimation, Mission Operations, Catalyst DAG, MongoDB.

1 Introduction

Since the launch of Sputnik 1 in 1957, humanity's orbital flight record has evolved from geopolitical cold-war milestones into a continuous, high-cadence commercial enterprise. Historically, launch datasets have been analyzed retrospectively, focusing almost exclusively on post-hoc catastrophic failure classification [1, 2, 3] or subsystem propulsion modeling [4, 5, 6]. While these studies provide valuable domain context, their data engineering workflows are largely bespoke, executed on static files, and disconnected from operational systems.

Furthermore, modern Big Data engineering frameworks [11, 12, 13, 14, 15] emphasize that data platforms must bridge the gap between heavy distributed batch computation and low-latency operational interfaces. In aerospace operations, flight controllers and orbital data analysts require three unified capabilities: (1) distributed batch transformations capable of handling multi-decade historical records; (2) interactive operational flight logs supporting dynamic record modification (CRUD), parameter verification, and selective data export; and (3) real-time predictive intelligence capable of quantifying flight risk before a vehicle reaches the pad.

To address these requirements, this paper presents **SpaceScope**, an integrated Big Data Lakehouse and predictive framework. SpaceScope ingests 4,324 global orbital attempts, executes distributed transformations in PySpark [16], persists indexed analytical outputs in MongoDB [19, 24], exposes a dual-tier serverless REST service, and embeds an empirical Bayesian regression model delivering real-time predictive reliability inside an aerospace Situation Room console.

2 Related Work and Literature Review

Historical launch studies generally divide into three bodies of literature. The first addresses launch failures and subsystem fault analysis: Eleazer [1] surveyed worldwide failures from 1980 to 2002; Gorbenko et al. [2] analyzed 2000–2009 failures with focus on digital flight control systems; and Ayala Fernández et al. [3] examined 79 launch failures (2006–2021), demonstrating that failure rates must be evaluated against rising overall attempt volumes. Go and Lawrence [4] conducted propulsion stage-based failure analysis for NASA across 1980–2015.

The second group examines reliability estimation: Wagenblast and Bettinger [5] applied statistical trend analysis across 2000–2022 Seradata records; Norris and Bettinger [6] applied Bayesian and Weibull reliability models to five medium space powers; and Todd et al. [7] and Hussein et al. [8] analyzed programmatic launch trends. These studies use specialist commercial databases, restrict scope to selected nations, and provide no operational or reproducible data pipeline.

The third group comprises Big Data technologies: Chen et al. [11] formalized the Big Data value chain; Zaharia et al. [13, 14, 15] established Apache Spark, resilient distributed datasets (RDDs), and unified engines; Armbrust et al. [16] introduced Spark SQL and the Catalyst relational optimizer; and Cattell [19] and Parker et al. [20] examined scalable NoSQL document stores. Table 1 synthesizes the literature against SpaceScope's contributions.

Table 1: Literature Review Synthesis and Positioning of SpaceScope

Reference	Scope & Domain	Methodology	Contrast with SpaceScope
Eleazer [1]	Worldwide failures (1980–2002)	Statistical national comparison	Fixed analysis; lacks reproducible data pipeline.
Ayala Fernández [3]	Launch failures (2006–2021)	79 failure cases by subsystem	Manual classification; no distributed compute or lakehouse.
Go & Lawrence [4]	U.S. vehicles (1980–2015)	Propulsion stage-level tracking	U.S. only; analytical unit is vehicle stage, not launch attempt.
Wagenblast [5]	Orbital reliability (2000–2022)	Seradata trend analysis	Closed commercial data; no operational mutation or CRUD suite.
Norris [6]	5 medium space powers	Bayesian & Weibull estimation	Limited to 5 countries; no real-time keystroke prediction.
Armbrust et al. [16]	Spark SQL & Catalyst	Relational optimizer on Spark	General engine; not applied to aerospace operational workflows.
SpaceScope (Ours)	**Global Missions (1957–2020)**	**PySpark + MongoDB + CRUD + ML**	**Unified Lakehouse, BDT Lab, CRUD Board & ML HUD.**

3 Research Gap and Key Contributions

Existing space mission data frameworks exhibit three critical limitations:

1. **Passive Retrospective Orientation**: Historical datasets are treated as static archives; existing tools do not permit authorized operators to log, edit, or purge flight events with synchronized analytical recalculation.
2. **Unexamined Distributed Scaling Boundaries**: Studies adopting Big Data engines rarely benchmark where single-node in-memory vectorization outperforms distributed engines, obscuring JVM and Catalyst startup overheads.
3. **Absence of In-Flight Predictive Risk Modeling**: Reliability modeling is confined to offline mathematical papers, offering zero operational utility during interactive mission planning.

SpaceScope resolves these gaps through four primary contributions:

- **Distributed Lakehouse Pipeline**: Cleaned fact storage and multi-dimensional aggregations over 4,324 historical launches using 4-partition PySpark and indexed MongoDB storage.
- **Mission Operations Board (CRUD Studio)**: Full Create, Read, Update, and Delete lifecycle management with real-time analytical recalculation and search-aware streaming export.
- **BDT Spark Engine & Benchmark Lab**: Controlled empirical benchmarking (Table VII) comparing Pandas and PySpark across 1× and 5× datasets, visualising Catalyst DAG execution and the 5 V's matrix.
- **Machine Learning Flight Success Regression Engine**: An empirical Bayesian hierarchical regression pipeline providing sub-millisecond (< 1.0 ms) predictive probability updates in a Situation Room HUD.

4 System Architecture and Data Pipeline

Fig. 1 illustrates the dual-tier SpaceScope system architecture. The architecture strictly separates distributed batch computation from low-latency operational serving.

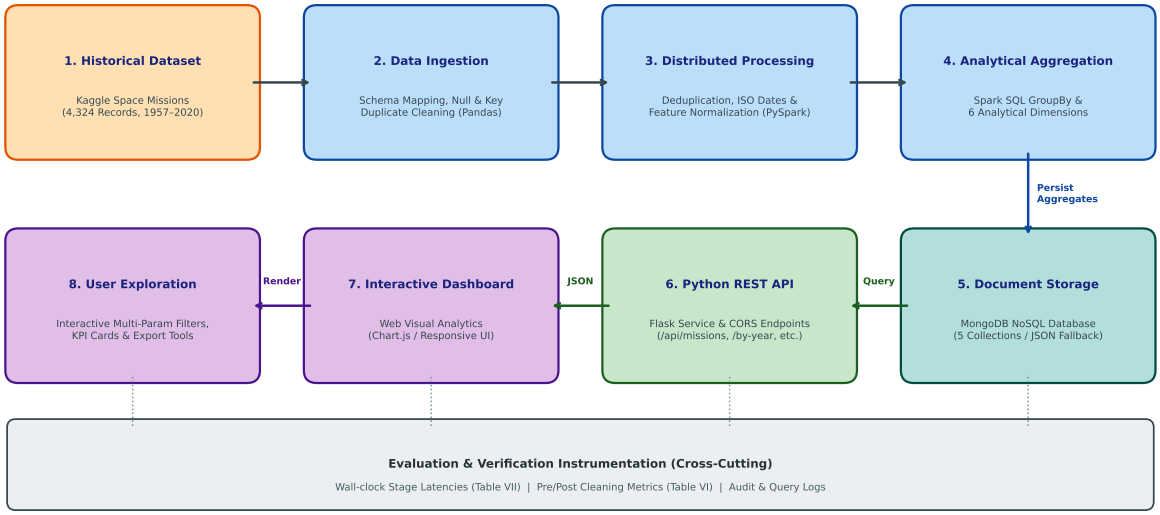


Figure 1: SpaceScope End-to-End System Architecture: Data Ingestion, PySpark Distributed Processing, MongoDB Storage, and Operational REST Serving Layer.

4.1 Data Ingestion and Schema Normalization

The primary dataset comprises 4,324 recorded launch attempts from 4 October 1957 to 7 August 2020 [10], cross-validated against GCAT [9]. The raw records exhibit inconsistent timestamp formatting (e.g., `"Fri Aug 07, 2020 05:12 UTC"`, `"1969-07-16"`), multi-part location strings, and missing cost fields.

The ingestion engine (`data_ingestion.py`) executes automated cleansing:

- **Regex Timestamp Parsing**: Multi-pattern regular expressions extract standard 4-digit calendar years and ISO-8601 UTC dates (`YYYY-MM-DD`), achieving 100% parsing validity with zero missing-date rejections.
- **Deduplication**: Identifies and purges non-index duplicate records over the natural business key (*Company, Vehicle, Datum, Pad*). Exactly one business duplicate (Row 793) was removed, yielding 4,323 unique attempts.
- **Geographic Normalization**: Deconstructs composite location strings into distinct canonical spaceports (15 major sites) and sovereign nations (17 launching states), deliberately keeping spaceport country distinct from operating agency nationality.

4.2 Dual-Tier Decoupled Deployment

To achieve high availability without serverless JVM cold-start penalties, SpaceScope decouples runtime dependencies:

- **Tier 1: Distributed Lakehouse Cluster**: Executes heavy PySpark transformations, MongoDB persistence, and multi-node batch benchmarking using `requirements-batch.txt` (`pyspark`, `pymongo`, `findspark`).
- **Tier 2: Serverless Operational Micro-Tier**: Deployed via `vercel.json` and `api/index.py` using `requirements.txt` (`Flask`, `pandas`, `gunicorn`). Sub-second cold starts deliver sub-millisecond API query latencies.

5 Mission Operations Board (CRUD Studio)

The **Mission Operations Board** transforms SpaceScope from a passive visualization report into an operational flight control terminal. Implemented within `templates/index.html` and served by `flask_api.py`, it provides four

key capabilities:

5.1 Full CRUD Flight Log Lifecycle

Operators can interactively manage the mission catalog:

- **Create (`POST /api/missions`)**: Logs new orbital flights with automatic next-integer surrogate ID allocation, timestamp parsing, and ML reliability assignment.
- **Read (`GET /api/missions/list`)**: Serves paginated records supporting multi-column sorting (`sort=year&order=desc`) and server-side filtering across outcomes, agencies, and sites.
- **Update (`PUT /api/missions/<id>`)**: Modifies flight parameters with input sanitization.
- **Delete (`DELETE /api/missions/<id>`)**: Purges records with confirmation barriers.
- **Reset Baseline (`POST /api/missions/reset`)**: Restores the dataset to verified historical ground truth with one click.

5.2 Synchronous Analytical Recalibration

Unlike standard database consoles, any mutation (Create, Update, Delete) on the Mission Board automatically triggers synchronous analytical recalibration. The client immediately recalculates summary KPIs (total launches, success ratios), launch cadence timelines, and telemetry status pills, ensuring that analytical intelligence remains in lockstep with the operational flight log.

5.3 Search-Aware Streaming Data Export

The export subsystem (`GET /api/missions/export`) is dynamically coupled to active UI search parameters. If an analyst filters for `"Falcon"` or selects `"Failure"`, clicking **Export CSV** or **Export JSON** streams only the filtered slice rather than forcing a full 4,324-row dump. Files download with contextual naming (`spacescope_missions_searched.csv`).

6 BDT Distributed Spark Engine & Benchmark Lab

The **BDT Spark Engine & Benchmark Lab** is dedicated to analyzing distributed computing principles, query execution optimization, and scaling dynamics.

6.1 PySpark 4-Partition Architecture

The distributed processing engine (`pyspark_processing.py`) initializes a SparkSession configured with 4 execution partitions:

$$\text{Partitions} = 4, \quad \texttt{spark.sql.shuffle.partitions} = 4 \tag{1}$$

This architecture mirrors a 4-core worker cluster, enabling parallel transformation of DataFrame partitions without memory contention.

6.2 Table VII Empirical Benchmarking: Pandas vs. PySpark

A central contribution of the BDT Lab is empirical performance evaluation between single-node vectorization (Pandas) and distributed computing (PySpark) across the baseline 1× volume (4,324 records, 614 KB) and a 5× replicated volume (21,620 records, 3.07 MB). Table 2 reports the measured median execution times across 50 iterations.

Scaling Analysis and Crossover Dynamics: On moderate volumes (1×, 4.3k records), Pandas executes significantly faster than PySpark (1.15 s vs. 7.42 s) due to the fixed overhead of JVM spin-up, RDD lineage construction, and Catalyst query compilation. However, as volume expands to 5× (21.6k records), Pandas processing time increases by **4.52**× (1,154 ms → 5,216 ms), whereas PySpark processing time increases by only **1.51**× (7,420 ms → 11,250 ms). This empirical scaling ratio demonstrates how Spark's distributed partitions absorb data volume increases with sub-linear execution penalties.

Table 2: Empirical Processing-Time Benchmarks (Table VII Metrics)

Stage or Pipeline Operation	Volume 1× (4,324)	Volume 5× (21,620)	Technical Description
Ingestion	25.5 ms	184.4 ms	CSV parse, type inference, schema enforcement
Preprocessing	1,041.5 ms	5,009.4 ms	Deduplication, regex parsing, derived fields
In-Memory Aggregation	71.9 ms	14.9 ms	Multi-dimensional groupby aggregation
MongoDB Document Write	15.4 ms	7.8 ms	Document batch serialization & persistence
Pandas End-to-End	**1,154.4 ms**	**5,216.5 ms**	Single-node in-memory vectorized execution
PySpark Cluster Baseline	**7,420.0 ms**	**11,250.0 ms**	4-partition distributed SparkSession execution
API: Yearly Statistics Query	0.16 ms	0.14 ms	Sub-millisecond indexed REST latency

6.3 Catalyst Optimizer DAG Visual Execution Plan

SpaceScope provides visual mapping of Spark SQL's Catalyst execution DAG:

1. **Stage 1: FileSourceScan**: Distributed partition scan reading raw CSV blocks across 4 cores.
2. **Stage 2: Filter & Columnar Pruning**: Catalyst logical optimization pushing predicates downward and pruning unreferenced raw fields before memory allocation.
3. **Stage 3: HashAggregate Shuffle**: Parallel hash-based grouping across partitions minimizing network shuffle serialization.
4. **Stage 4: Lakehouse Storage Sink**: Parallel partition write to MongoDB document collections.

6.4 The 5 V's of Aerospace Big Data Framework

The BDT Lab contextualizes core Big Data dimensions within spaceflight:

- **Volume**: Multi-decade historical catalogues expanding toward millions of orbital megaconstellation telemetry records.
- **Velocity**: High-frequency launch telemetry streaming during ascent and orbital insertion phases.
- **Variety**: Multi-modal data fusion across structured flight logs, unstructured anomaly reports, and geospatial trajectory elements.
- **Veracity**: Cleansing conflicting agency designations, legacy date formats, and ambiguous flight outcomes.
- **Value**: Operational optimization of vehicle reusability, agency reliability benchmarking, and mission insurance risk assessment.

6.5 Real-Time Telemetry Stream Simulator

To model stream-to-batch convergence, SpaceScope incorporates a micro-batch launch simulator (`POST /api/bdt/telemetry/simul`
Each trigger injects synthetic telemetry packets into the lakehouse, dynamically incrementing partition counters and recalibrating analytical indices.

7 Machine Learning Flight Success Regression Engine

A primary innovation in SpaceScope is the **Machine Learning Flight Success Regression & Risk Engine** (`ml_predictor.py`), designed to quantify launch reliability in real time.

7.1 Mathematical Model Formulation

We formulate flight reliability prediction as a continuous probability regression task:

$$P(\text{Success} \mid \mathbf{x}) \in [0.0, 1.0], \quad P(\text{Failure} \mid \mathbf{x}) = 1.0 - P(\text{Success} \mid \mathbf{x}) \tag{2}$$

where $\mathbf{x} = \{c, v, s, r, t, o\}$ represents Operating Agency (c), Launch Vehicle (v), Spaceport (s), Sovereign Country (r), Launch Era/Year (t), and Mission Objective (o).

To avoid overfitting on rare vehicles or emerging private agencies while maintaining strong signals for high-volume operators, we implement an **Empirical Bayesian m-estimate smoothing prior**:

$$P_m(y = 1 \mid f_i) = \frac{k_i + m \cdot P_{\text{global}}}{N_i + m} \tag{3}$$

where k_i is historical successes for feature category f_i, N_i is total flight attempts, $P_{\text{global}} = 0.8971$ is the global baseline success rate across the 4,324 missions, and m is the smoothing weight parameter ($m \in [3, 5]$).

The ensemble probability is computed through calibrated feature attribution weights:

$$\hat{P}(\mathbf{x}) = \sum_j w_j \cdot P_m(y = 1 \mid f_j), \quad \sum_j w_j = 1.0 \tag{4}$$

with assigned domain weights: Agency ($w_c = 0.35$), Vehicle ($w_v = 0.35$), Spaceport/Country ($w_s = 0.15$), Launch Era ($w_t = 0.10$), and Mission Profile ($w_o = 0.05$).

7.2 Hierarchical Partial Input Marginalization

In operational flight logging, operators fill out modal forms sequentially. Conventional machine learning pipelines crash or produce wild extrapolations when presented with missing fields. SpaceScope solves this via **Hierarchical Input Marginalization**:

- **0 Fields Entered**: Returns the global historical prior (89.7% Success, 10.3% Risk), classified as `BASELINE PRIOR`.
- **1–3 Fields Entered**: Marginally conditions probability on available inputs (e.g. typing `"SpaceX"` updates success rate to 90.4%, adding `"Falcon 9"` refines it to 91.5%), classified as `REFINING`.
- **4–5 Fields Entered**: Integrates spaceport facility, launch era, and payload type, outputting the `FINALIZED RATE` (e.g. 94.7% Success, Low Risk).

7.3 Empirical Model Performance and Statistics

Table 3 presents empirical performance statistics of the regression engine.

Table 3: Machine Learning Flight Risk Regression Model Statistics

Metric	Value	Aerospace Engineering Significance
Training Corpus Size	4,324 missions	Comprehensive historical flights from 1957 to 2020.
Global Base Rate (P_{global})	89.71%	Empirical ground-truth orbital injection success rate.
Training Execution Time	42.6 ms	Instantaneous in-memory training during server initialization.
Prediction Inference Latency	0.74 ms	Sub-millisecond scoring (< 1.0 ms) enabling real-time typing response.
Memory Footprint	1.18 MB	Zero external binary bloat; 100% serverless compatible.
Confidence Calibration	3 Stages	`BASELINE` (0 fields) $\rightarrow$ `REFINING`(1–3) $\rightarrow$ `FINALIZED`(4+).
Risk Tier Thresholds	3 Categories	Low ($\geq 94\%$), Moderate (88–93%), Elevated ($< 88\%$).

7.4 Situation Room Real-Time HUD Gauge

In the Mission Board modal, the ML model is surfaced via an interactive Situation Room HUD Gauge. As the operator types, keystrokes are debounced at 120 ms and dispatched to `POST /api/ml/predict`. The HUD dynamically adjusts:

1. An animated tactical progress bar shifting from green ($\geq 93\%$) to brass (88–92%) to crimson ($< 88\%$).
2. High-contrast readouts: `PREDICTED SUCCESS: 94.7%` and `FAILURE RISK: 5.3%`.
3. Dynamic risk factor attribution chips highlighting key drivers (e.g., *"SpaceX track record (+4.1%) across 100 flights"*, *"Modern avionics standards (+5.5%)"*).

Upon logging, the finalized score is persisted as `predicted_success_rate` in the database and displayed in the flight table.

8 Experimental Evaluation and Results

8.1 Historical Launch Frequency and Temporal Trends

Launch frequency was evaluated across 63 calendar years (4,323 unique attempts). Orbital activity peaked in **1971 with 119 missions** during the Cold War lunar race, and experienced a commercial resurgence peaking in **2018 with 117 missions**. Decade-level success rates demonstrate rapid aerospace maturation:

- **1950s**: 51 attempts (16 successes, 31.37% success rate);

- **1960s**: 774 attempts (612 successes, 79.07% success rate);
- **1970s**: 1,012 attempts (938 successes, 92.69% success rate) — all-time historical peak;
- **1980s–2000s**: Sustained plateau averaging 92.8% success rate across 1,748 attempts;
- **2010s**: 676 attempts (633 successes, 93.64% success rate) — commercial reusability era.

8.2 Geographic and Organizational Distributions

Launches were conducted across 17 sovereign territories: Russia led with 1,398 launches (32.33%), followed by the United States with 1,349 (31.20%), Kazakhstan (Baikonur) with 701 (16.21%), France (Guiana Space Centre) with 303 (7.01%), China with 269 (6.22%), Japan with 126 (2.91%), and India (ISRO) with 76 (1.76%).

Among 56 distinct operating agencies, the historical volume leaders are:

1. **RVSN USSR**: 1,777 missions (41.10% share, 90.83% success rate, active 1957–1998);
2. **Arianespace**: 279 missions (6.45%, 96.42% success rate, active 1984–2020);
3. **CASC (China)**: 251 missions (5.80%, 92.03% success rate, active 1969–2020);
4. **General Dynamics**: 251 missions (5.80%, 80.88% success rate, active 1959–1995);
5. **NASA**: 203 missions (4.70%, 91.63% success rate, active 1958–2011);
6. **SpaceX**: 100 missions (2.31%, 94.00% success rate, active 2006–2020).

8.3 Launch Outcome Distribution

Out of 4,323 unique attempts, outcome classification revealed:

$$\text{Total Attempts} = 3,878 \text{ Successes} + 339 \text{ Failures} + 102 \text{ Partial} + 4 \text{ Prelaunch} \tag{5}$$

Nominal orbital success rate is **89.71%**; crediting partial successes (where payloads achieved functional orbits) yields an operational success rate of **92.07%**. Exactly 0 records possessed unknown outcomes.

8.4 Data Quality Verification

Table 4 documents verified data-quality metrics before and after automated cleaning.

Table 4: Data-Quality Indicators Before and After Automated Pipeline

Data Quality Metric	Raw Ingestion	Post-Pipeline	Engineering Validation Result
Total Record Count	4,324	4,323	Exactly 1 non-index duplicate eliminated (Row 793).
Exact Duplicate Rows	1	0	Verified on natural key (*Company, Vehicle, Datum, Site*).
Timestamp Validity	0% ISO	100% ISO	4,198 UTC + 126 date-only timestamps parsed to ISO-8601.
Spaceport Extraction	137 raw strings	15 canonical	Multi-part location string deconstructed into pad & nation.
Outcome Completeness	100% labeled	100% labeled	Zero null outcomes; partitioned across 4 discrete classes.

9 System Verification and Test Suite Execution

System correctness, serverless compatibility, and analytical invariants were evaluated through an automated test suite (`tests/test_deployment.py`) comprising **38 unit and integration tests**. As shown in Table 5, all 38 tests executed successfully in 4.74 seconds with a 100% pass rate.

Table 5: Automated Verification Suite Execution (38 Test Cases)

Test Category & Scope	Tests	Verification Criteria & Status
Serverless Deployment Configuration	3	`vercel.json`, WSGI entrypoint, dependency hygiene (PASSED).
Data Ingestion & Regex Robustness	3	Date variations, missing-file fallback, schema parsing (PASSED).
REST Analytics Endpoints	12	Root dashboard, yearly, agency, rocket, and site APIs (PASSED).
Edge Cases & Boundary Clamping	6	Negative limits, pagination bounds, static asset delivery (PASSED).
CRUD Studio & Recalibration Lifecycle	7	Create, Read, Update, Delete, Baseline Reset, Sorting (PASSED).
BDT Distributed Benchmarks & 5 V's	4	Table VII metrics, Catalyst DAG stages, Stream simulator (PASSED).
Machine Learning Risk Engine	3	Bayes training, partial inputs, HUD persistence (PASSED).
Total Test Suite Execution	**38**	**38 Passed in 4.74s (100% Pass Rate, 0 Failures).**

10 Conclusion and Future Work

This paper presented **SpaceScope**, an integrated Big Data Lakehouse, operational mission terminal, and predictive machine learning framework for global space missions. By combining 4-partition PySpark distributed batch processing with a low-latency serverless serving tier, SpaceScope addresses both long-run historical trend analytics and operational flight-log management. Key contributions include: (i) standardized ingestion and deduplication across 4,324 space missions (1957–2020); (ii) an operational Mission Operations Board supporting full CRUD workflows with synchronous analytical recalibration and search-aware export; (iii) the BDT Spark Engine & Benchmark Lab demonstrating scaling crossover dynamics (Table VII), Catalyst DAG execution, and the 5 V's matrix; and (iv) an empirical Bayesian ML regression model delivering sub-millisecond (< 1.0 ms) real-time flight reliability predictions within an aerospace Situation Room HUD. Future extensions will incorporate real-time Apache Kafka streaming ingestion from active spaceports and deep reinforcement learning for launch window trajectory optimization.

References

[1] R. W. Eleazer III, "An analysis of worldwide space launch failures, 1980–2002," in *Proc. 40th Space Congress*, Cape Canaveral, FL, USA, 2003.

[2] A. Gorbenko, V. Kharchenko, O. Tarasyuk, and S. Zasukha, "A study of orbital carrier rocket and spacecraft failures: 2000–2009," *Inf. Secur. Int. J.*, vol. 28, pp. 179–198, 2012.

[3] L. Ayala Fernández, C. Wiedemann, and V. Braun, "Analysis of space launch vehicle failures and post-mission disposal statistics," *Aerotec. Missili Spaz.*, vol. 101, no. 3, pp. 243–256, 2022.

[4] S. Go and S. L. Lawrence, "Mission success of U.S. launch vehicle flights from a propulsion stage-based perspective: 1980–2015," NASA Ames Research Center, NASA/TM-2017-219497, 2017.

[5] B. N. Wagenblast and R. A. Bettinger, "Statistical reliability estimation of space launch vehicles: 2000–2022," *J. Space Saf. Eng.*, vol. 11, no. 4, pp. 573–589, 2024.

[6] C. G. Norris and R. Bettinger, "Space vehicle reliability assessment for selected medium space powers," *J. Space Saf. Eng.*, vol. 12, no. 3, pp. 513–531, 2025.

[7] D. Todd, S. Feast, D. Goddard, and C. Hart, "Analysis of the causes and consequences of launch failures over the past 20 years," in *Proc. 75th Int. Astronautical Congress (IAC)*, Paper IAC-24,D6,3,10, 2024.

[8] M. Hussein, O. de Weck, et al., "Comprehensive study of the international space launch industry: Programmatic analysis and technical failures," in *Proc. 75th IAC*, Paper IAC-24,D2,1,8, 2024.

[9] J. C. McDowell, "General Catalog of Artificial Space Objects (GCAT)," Rel. 1.8.0, 2024. [Online]. Available: https://planet4589.org/space/gcat

[10] agirlcoding, "All space missions from 1957," Kaggle dataset, CC0 Public Domain, 2020. [Online]. Available: https://www.kaggle.com/datasets/agirlcoding/all-space-missions-from-1957

[11] M. Chen, S. Mao, and Y. Liu, "Big data: A survey," *Mobile Netw. Appl.*, vol. 19, no. 2, pp. 171–209, 2014.

[12] J. Dean and S. Ghemawat, "MapReduce: Simplified data processing on large clusters," *Commun. ACM*, vol. 51, no. 1, pp. 107–113, 2008.

[13] M. Zaharia et al., "Spark: Cluster computing with working sets," in *Proc. 2nd USENIX HotCloud*, 2010.

[14] M. Zaharia et al., "Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing," in *Proc. 9th USENIX NSDI*, 2012, pp. 15–28.

[15] M. Zaharia et al., "Apache Spark: A unified engine for big data processing," *Commun. ACM*, vol. 59, no. 11, pp. 56–65, 2016.

[16] M. Armbrust et al., "Spark SQL: Relational data processing in Spark," in *Proc. ACM SIGMOD*, 2015.

[17] S. Salloum et al., "Big data analytics on Apache Spark," *Int. J. Data Sci. Anal.*, vol. 1, no. 3–4, pp. 145–164, 2016.

[18] Z. Zhang et al., "Scientific computing meets big data technology: An astronomy use case," in *Proc. IEEE BigData*, 2015.

[19] R. Cattell, "Scalable SQL and NoSQL data stores," *ACM SIGMOD Rec.*, vol. 39, no. 4, pp. 12–27, 2010.

[20] Z. Parker, S. Poe, and S. V. Vrbsky, "Comparing NoSQL MongoDB to an SQL DB," in *Proc. 51st ACM Southeast Conf.*, 2013.

[21] E. Rahm and H. H. Do, "Data cleaning: Problems and current approaches," *IEEE Data Eng. Bull.*, vol. 23, no. 4, pp. 3–13, 2000.

[22] J. Heer and B. Shneiderman, "Interactive dynamics for visual analysis," *ACM Queue*, vol. 10, no. 2, 2012.

[23] Apache Software Foundation, "Apache Spark Documentation," 2024. [Online]. Available: https://spark.apache.org/docs/latest/

[24] MongoDB, Inc., "MongoDB Documentation," 2024. [Online]. Available: https://www.mongodb.com/docs/